VULCANO

SUSTENTO TÉCNICO DEL SPRINT DE AUTENTICACIÓN Y SEGURIDAD

Módulo: Login y administración de sesiones

Control documental	Valor
Versión	1.0
Fecha de elaboración	27 de octubre de 2026
Responsable	[Campo editable: nombre y cargo]
Estado	[Campo editable: revisión / aprobación]
Corte técnico	Árbol de trabajo inspeccionado el 27/10/2026
Alcance de evidencia	Código frontend, OpenAPI local y pruebas automatizadas; sin auditoría de backend o prueba de login real

Documento de transferencia técnica y sustento del sprint. Las capacidades pendientes se identifican expresamente; no constituye certificación de seguridad o autorización de despliegue.

Identidad visual documental: fondo claro, encabezados oscuros y acento oliva derivados de la interfaz existente. No se modifica el diseño de la aplicación.

Contenido y lectura de estados

1. Resumen ejecutivo

2. Antecedentes y justificación

3. Objetivo general y objetivos específicos

4. Alcance del sprint

5. Arquitectura técnica

6. Desarrollo del módulo de login

7. Implementación de seguridad

8. Integración frontend–backend

9. Documentación del código fuente

10. Historias de usuario y criterios de aceptación

11. Pruebas y validación

12. Resultados y avances del sprint

13. Riesgos y actividades pendientes

14. Conclusiones

15. Anexos

Estado	Criterio de uso
Implementado y validado	Código existente y caso ejecutado aplicable; indicar cuando se usa backend simulado.
Implementado sin validar	Código/entregable presente, sin ejecución de su comprobación correspondiente.
Parcialmente implementado	Parte del requisito funciona; la integración o validación restante se identifica.
Pendiente	Sin implementación del requisito en el frontend revisado.
No verificable	Falta evidencia del servidor o ejecución real; no se infiere a partir del requerimiento.

1. Resumen ejecutivo

El sprint consolidó el login existente de VULCANO mediante un servicio de sesión centralizado, interfaces de bootstrap, validación previa a la autenticación, catálogos de sesión y cierre local con aislamiento de respuestas y vistas privadas. Las verificaciones actuales confirman 53 pruebas enfocadas aprobadas y compilación de desarrollo correcta. La suite completa presenta 166 pruebas aprobadas y 10 fallidas; producción no supera el presupuesto de estilos de Hoja Milimetrada.

La plataforma aún no integra los endpoints /api/v1/auth/*. El código utiliza POST /api/auth/token traducido a /token por el proxy de desarrollo. El OpenAPI local consultado publica /token pero no las seis rutas nuevas. No se efectuaron solicitudes de login con credenciales reales, renovación, revocación o desbloqueo. La política de permisos numéricos existe como funciones puras probadas, sin conexión a menús, rutas o acciones.

El estado global del sprint es parcialmente implementado. Refresh rotativo, sesión deslizante, recuperación /yo, revocación Redis y administración de IPs continúan pendientes de contrato e implementación. Este informe distingue los avances verificables de esos objetivos para evitar dar por terminadas capacidades que todavía no operan.

2. Antecedentes y justificación

El contrato de autenticación evolucionó desde campos opcionales antiguos hacia una respuesta bootstrap con usuario y catálogos. Aceptar solo un token sin validar el resto podía dejar datos incompletos para la navegación. Centralizar validación y sesión reduce ese riesgo y evita que varios componentes creen copias inconsistentes de identidad.

Los riesgos abordados son uso local de credenciales vencidas, reconstrucción de sesión por una respuesta tardía después de salir, reutilización de datos privados entre usuarios y exposición del token a destinos externos. La autorización detallada por función, revocación remota y prevención de fuerza bruta requieren aún coordinación con el contrato/backend; no se presume su cobertura.

3. Objetivo general y objetivos específicos

Sustentar técnicamente el estado del módulo de login y administración de sesiones, con trazabilidad desde requisito hasta código y verificación ejecutada.

- Identificar archivos, contratos, dependencias y consumidores de identidad.
- Explicar el flujo real del bootstrap, expiración y cierre local.
- Separar autenticación operativa de la política numérica preparada y de la arquitectura v1 futura.
- Registrar resultados de pruebas/builds y hallazgos priorizados sin cambiar comportamiento.
- Entregar Word y anexos mantenibles para transferencia del conocimiento.

4. Alcance del sprint

Incluido y observado	Excluido / pendiente
Formulario existente, URL-encoded, mensajes seguros y validación integral.	Sin reconstrucción del login ni cambio visual.
Estado en memoria, expiración por expira_en y catálogos.	Sin persistencia de refresh ni recuperación /yo.
Guards de autenticación, límites de Bearer y logout local.	Sin revocación remota ni validación criptográfica Angular.
Política numérica probada en aislamiento.	Sin IDs asociados a funciones ni aplicación en menús/rutas/acciones.
Comentarios TSDoc, documentación y pruebas.	Sin cambios funcionales a mapas, cálculos, backend o herramientas operacionales.

5. Arquitectura técnica

La arquitectura usa componentes standalone Angular, servicios providedIn:root, signals y RxJS. AuthService centraliza tokens, usuario, expiración y generación de sesión; CatalogosService mantiene el bootstrap sin HTTP adicional. LoginComponent y Sidebar consumen esa fuente. AppShell registra limpieza de herramientas privadas y mantiene PuntosDemoService por instancia.

Tecnología instalada	Versión comprobada
@angular/core	21.2.19
@angular/cli	21.2.20
rxjs	7.8.2
typescript	5.9.3
vitest	4.1.10

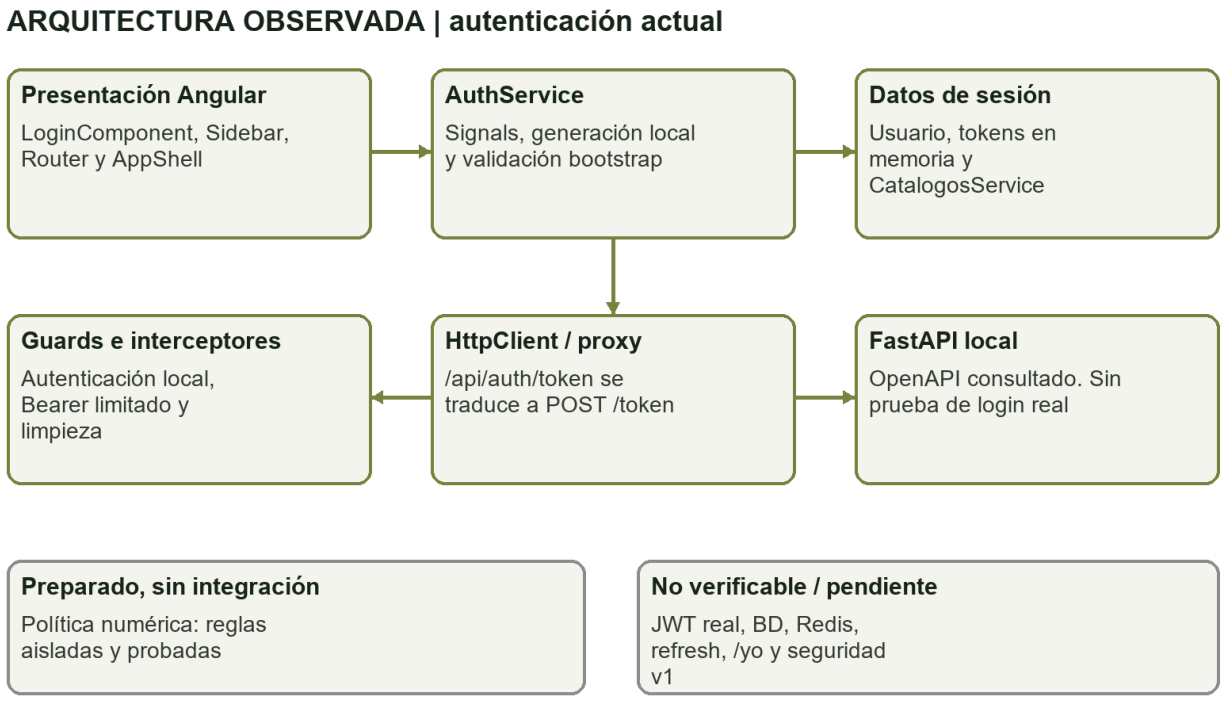
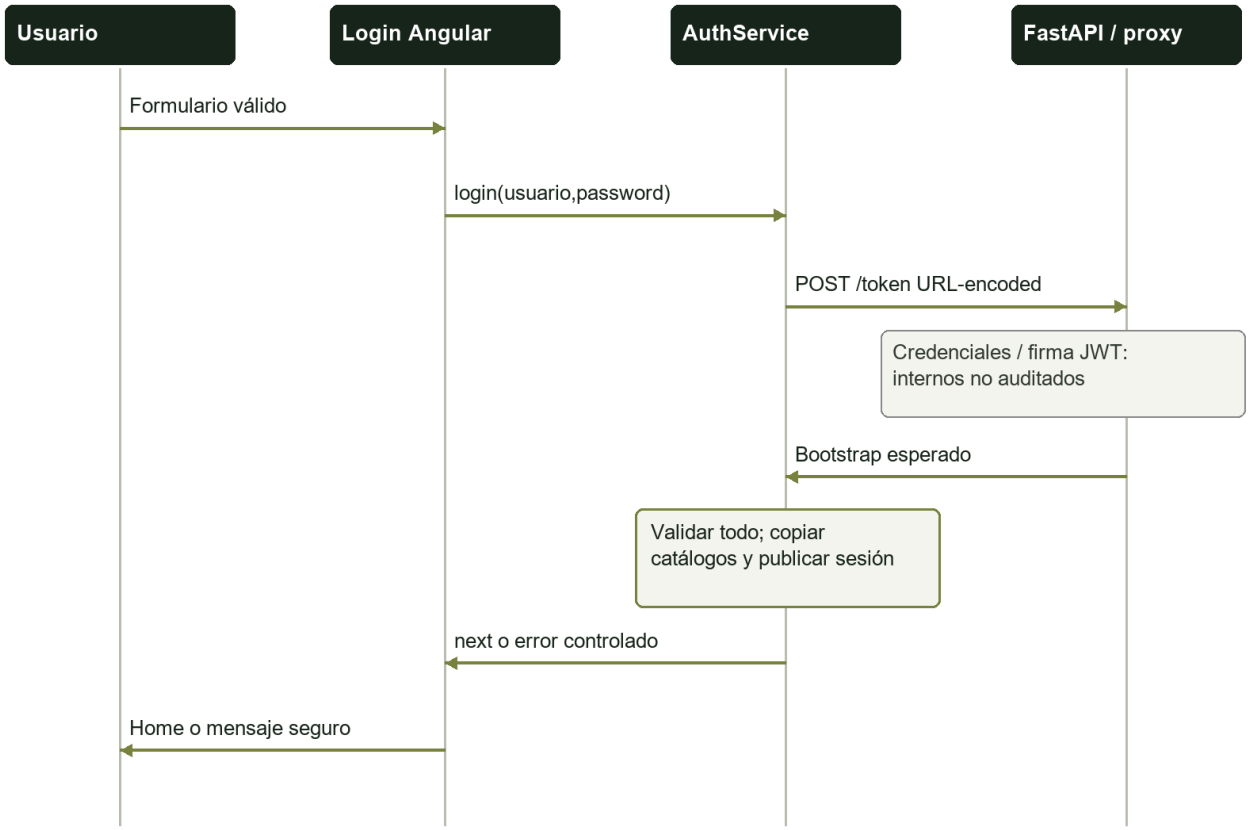


Figura 1. Arquitectura observada. Los bloques inferiores son preparación o aspectos no verificables; no se representan como integración operativa.

FastAPI se acredita por el OpenAPI local de “Servidor VULCANO”, versión 2.0.0. No se inspeccionó su código fuente de autenticación ni la infraestructura. La generación/verificación JWT, base de datos de usuarios/roles/permisos y Redis se describen como responsabilidades esperadas del servidor, no como implementación comprobada.



No hay refresh, /yo, rotación o logout remoto en este flujo.

Figura 2. Secuencia real esperada por el frontend y simulada en pruebas. El contrato HTTP está publicado; el éxito autenticado real no fue ejecutado.

5.1 Límites de confianza

El JSON HTTP no se considera válido por el tipo TypeScript: validarLogin comprueba todos los campos antes de publicar la sesión. Un token presente en memoria no demuestra firma ni autorización remota. El servidor debe verificar autenticidad, revocación y autorización. El frontend no contiene clave secreta JWT ni intenta verificar con ella.

6. Desarrollo del módulo de login

1. El formulario requiere usuario y contraseña. El envío se detiene si ya existe una petición pendiente o controles inválidos. No se implementa un segundo login.
2. AuthService.login devuelve un Observable diferido, cancela el intento previo y envía username/password URL-encoded a ApiConfigService.loginUrl. HttpContext excluye Bearer previo y el logout global por error de credenciales.
3. El timeout configurado es 30000 ms. validarLogin exige access_token/refresh_token no vacíos, tipo_token bearer, expira_en positivo y finito, usuario completo y cinco catálogos con registros compatibles.
4. El servicio copia usuario y catálogos; calcula expiresAt con Date.now()+expira_en*1000. Publica el access token tras cargar catálogos. No usa token_type/expires_in, no decodifica claims y no consulta perfil adicional.
5. El componente navega a /home solo al recibir next válido. finalize libera el estado de envío; takeUntilDestroyed cancela al destruir la vista. Los errores no exponen contenido arbitrario del servidor.
6. Logout incrementa la generación, limpia estado/catálogos, cancela login pendiente, retira claves propias antiguas y ejecuta callbacks privados antes de redirigir a /login con replaceUrl. La promesa concurrente es compartida. No se comunica con el servidor.

6.1 Bootstrap y catálogos

Campo / conjunto	Tratamiento actual
access_token / refresh_token	Memoria; solo access se usa en Authorization. Refresh reservado para integración futura.
tipo_token / expira_en	Se exige bearer; expira_en determina vencimiento local.
usuario	Identidad, grado, correo, rol, permisos textuales, aprobación de documentos y foto opcional.
tipos_unidad / unidades	Almacenamiento en CatalogosService; jerarquía mediante unidad_padre_id.
estados_operacionales / estados_logisticos / permisos	Catálogos disponibles sin nuevas consultas; permisos del catálogo no son asignaciones.

La fotografía relativa se resuelve contra el origen auth configurado y solo admite HTTP(S). Sin foto retorna null. Sidebar muestra grado y nombre completo. El código preparado de NavigationService no se encontró instanciado, por lo que no se afirma trazabilidad activa de navegación.

7. Implementación de seguridad

7.1 Tokens y claims

El frontend trata los tokens como cadenas opacas y no implementa JwtPayload ni lee claims. sub, rol, permisos:number[], exp, iat y jti son claims requeridos para la futura integración, pendientes de comprobar en un JWT real anonimizado. expira_en del bootstrap es la única base actual de expiración. No se afirma que los tokens emitidos realmente contengan esos claims o que una firma se haya validado en esta revisión.

Claim requerido	Estado de evidencia frontend
sub	Sin lectura/validación implementada; confirmar contrato real
rol: ADMINISTRADOR | USUARIO	Sin lectura/validación implementada; confirmar contrato real
permisos: number[]	Sin lectura/validación implementada; confirmar contrato real
exp	Sin lectura/validación implementada; confirmar contrato real
iat (opcional)	Sin lectura/validación implementada; confirmar contrato real
jti (opcional)	Sin lectura/validación implementada; confirmar contrato real

7.2 Matriz de roles y permisos

Rol / asignaciones	Política numérica preparada	Estado operacional
ADMINISTRADOR / []	Permite cualquier ID válido	No integrada. AuthService textual no concede excepción.
ADMINISTRADOR / lista	Solo IDs de la lista	No integrada a funciones.
USUARIO / []	No concede acceso; conserva autenticado el contexto	Sin guard de función ni alerta específica implementada.
USUARIO / lista	Solo IDs de la lista	Pendientes seed y enlaces menú/ruta/acción.
Sin sesión / rol desconocido	Deniega	authGuard sí exige sesión, pero no aplica la política numérica.

Los IDs de catálogo no equivalen a permisos asignados. AuthService.tienePermiso(codigo:string) verifica una lista textual del usuario, no todos los registros del catálogo. Sidebar.canShowItem usa un booleano estático; permisoCodigo no se utiliza para autorizar. Las funciones puras tienen cobertura aislada y no deben presentarse como control operacional completo.

7.3 Sesión deslizante y renovación

Access token de 25 minutos e inactividad de 60 minutos son requisitos nuevos. No existe contador de actividad humana, renovación periódica, refresh compartido, rotación ni reintento de 401. El código solo comprueba expiración al consultar autenticación/token o ejecutar guards. No hay renovación infinita por tráfico automático porque no hay renovación implementada. El siguiente sprint debe medir interacciones reales, no telemetría o teselas, y respetar el vencimiento impuesto por el servidor.

7.4 Persistencia, cierre y revocación

Tokens y usuario están en memoria, reduciendo su permanencia en almacenamiento del navegador sin eliminar el riesgo de XSS. Recargar implica nuevo login. Se retiran claves históricas propias de ambos almacenamientos; otras preferencias permanecen. Cookies HttpOnly no pueden limpiarse desde este código. Logout no confirma revocación de access/refresh en Redis; tampoco se consultó Redis o una base de datos. No hay mecanismo comprobado de lista de revocación o control de fuerza bruta del servidor.

7.5 Guards, interceptores y errores

authGuard protege rutas del shell y guestGuard evita abrir login con sesión activa. authInterceptor adjunta access solo a bases autorizadas y filtra respuestas exitosas obsoletas. errorInterceptor conserva sesión ante 403, cierra

localmente ante 401 protegido actual y propaga errores. Los 5xx se registran completos: se identifica oportunidad de redacción segura de logs. No existe guard administrador, panel de bloqueos o autorización por IDs en acciones internas.

8. Integración frontend–backend

Métodos y esquemas se documentan solo cuando tienen evidencia. La existencia de una URL configurada no prueba un consumidor implementado. El OpenAPI consultado no enumera /api/v1/auth/*; su ausencia en ese documento no demuestra que no exista otro servidor o versión. No se probaron credenciales ni operaciones administrativas reales.

/api/auth/token → /token

Propiedad	Evidencia actual
Método real	POST confirmado
Entrada / salida	username/password URL-encoded; respuesta esperada LoginResponse: access_token, refresh_token, tipo_token, expira_en, usuario, catalogos.
Autenticación	Sin Bearer de sesión anterior; contextos excluyen auth/error.
Consumidor	AuthService.loginWithBackend → LoginComponent
Respuestas y errores	OpenAPI local declara 200 (esquema libre) y 422. 401/400, red y 5xx cubiertos por manejo/pruebas HTTP simuladas, no verificados en servidor.
Estado	Parcialmente implementado
Sustento	Código y OpenAPI local; pruebas con HttpTestingController. No login real ejecutado.

/api/v1/auth/login

Propiedad	Evidencia actual
Método real	No confirmado
Entrada / salida	Bootstrap solicitado; método, cuerpo y esquema real pendientes.
Autenticación	No confirmada
Consumidor	Ninguno para v1
Respuestas y errores	No confirmados
Estado	Pendiente
Sustento	Ruta en requerimiento; ausente del OpenAPI local consultado.

/api/v1/auth/refresh

Propiedad	Evidencia actual
Método real	No confirmado
Entrada / salida	Transporte del refresh y respuesta de rotación pendientes.
Autenticación	No confirmada
Consumidor	Ninguno
Respuestas y errores	No confirmados
Estado	Pendiente
Sustento	Refresh conservado en memoria, sin llamada ni operación compartida.

/api/v1/auth/logout

Propiedad	Evidencia actual
Método real	No confirmado
Entrada / salida	Cuerpo y confirmación de revocación pendientes.
Autenticación	Access token vigente solicitado; contrato no verificado

Propiedad	Evidencia actual
Consumidor	Ninguno remoto; logout actual es local
Respuestas y errores	No confirmados
Estado	Pendiente
Sustento	No hay HttpClient hacia logout en AuthService.

/api/v1/auth/yo

Propiedad	Evidencia actual
Método real	No confirmado
Entrada / salida	Esquema del perfil y recuperación pendientes.
Autenticación	No confirmada
Consumidor	Ninguno
Respuestas y errores	No confirmados
Estado	Pendiente
Sustento	Sin recuperación de sesión persistida.

/api/v1/auth/seguridad

Propiedad	Evidencia actual
Método real	No confirmado
Entrada / salida	Estructura de intentos/IPs/Redis pendiente.
Autenticación	Administrador requerido por especificación, no integración comprobada
Consumidor	Ninguno
Respuestas y errores	No confirmados
Estado	Pendiente
Sustento	Sin componente ni guard de administrador.

/api/v1/auth/seguridad/bloqueo/{ip}

Propiedad	Evidencia actual
Método real	No confirmado
Entrada / salida	IP, cuerpo, método y salida pendientes; confirmación manual requerida.
Autenticación	Administrador requerido por especificación
Consumidor	Ninguno
Respuestas y errores	No confirmados
Estado	Pendiente
Sustento	No se enviaron solicitudes de desbloqueo.

Configuradas pero no consumidas: /api/auth/logout y /api/auth/refresh. No deben confundirse con las rutas v1. La semántica de seguridad requerida por el usuario (revocación Redis, bloqueo temporal, intentos fallidos y desbloqueo manual) es una dependencia de contrato, no una función demostrada por el OpenAPI consultado.

9. Documentación del código fuente

Se añadieron comentarios y TSDoc en español sin alterar firmas o instrucciones ejecutables. La comparación SHA-256 del JavaScript transpilado con removeComments=true confirma igualdad en 15/15 archivos. Esta comprobación complementa, pero no sustituye, las pruebas del comportamiento.

Grupo	Archivos comentados en este corte
Autenticación	auth.service.ts; auth-response.validator.ts; catalogos.service.ts; authorization-policy.ts; auth.guard.ts; guest.guard.ts
HTTP y servicios	auth.interceptor.ts; error.interceptor.ts; api-config.service.ts; navigation.service.ts
Consumidores	login.component.ts; sidebar.ts; app-shell.ts

El conjunto de comparación también incluye auth.models.ts y auth-http-context.ts, revisados sin necesitar comentarios adicionales. Los cambios anteriores del árbol ya estaban presentes al iniciar esta documentación: no se atribuyen a esta fase ni a un autor sin evidencia.

El inventario detallado de funciones, parámetros, retornos, errores y dependencias se incorpora en el anexo 15.1 y en INVENTARIO_ARCHIVOS_AUTENTICACION.md. Las firmas se extraen del AST; interfaces, plantillas y configuración no se presentan como métodos ejecutables.

10. Historias de usuario y criterios de aceptación

HU-01 — Como usuario necesito iniciar sesión con bootstrap válido.

Criterios de aceptación: Formulario URL-encoded; solo autenticar con tokens/usuario/catálogos completos; credenciales incorrectas dejan formulario utilizable.

Estado: Implementado y validado

Evidencia y límites: login.component.spec.ts; login-contract.spec.ts; api-connection.spec.ts. Validación con backend simulado.

HU-02 — Como usuario necesito renovar la sesión mientras mantengo actividad.

Criterios de aceptación: Access 25 min, ventana 60 min, rotación única compartida y sin extender sesión por telemetría.

Estado: Pendiente

Evidencia y límites: No hay refresh, listeners de actividad ni temporizador; 25/60 min son requisitos, no valores operativos verificados.

HU-03 — Como administrador necesito limitar funciones mediante roles y permisos.

Criterios de aceptación: ADMINISTRADOR vacío permite todo; otras listas limitan; aplicar a menú, ruta y acción con IDs documentados.

Estado: Parcialmente implementado

Evidencia y límites: authorization-policy.spec.ts valida funciones puras; no conexión operacional. AuthService sigue usando string[].

HU-04 — Como usuario necesito cerrar mi sesión e impedir reutilización local.

Criterios de aceptación: Limpiar tokens/catálogos, cancelar login pendiente, descartar respuestas anteriores y destruir vistas privadas; revocación remota diferenciada.

Estado: Parcialmente implementado

Evidencia y límites: logout.spec.ts y app-shell.spec.ts validan cierre local. Redis/revocación pendientes.

HU-05 — Como administrador necesito consultar y desbloquear IPs autorizadas.

Criterios de aceptación: Solo administrador; contrato confirmado; mostrar IP; confirmación manual y refrescar lista tras éxito.

Estado: Pendiente

Evidencia y límites: Sin módulo ni consumidor HTTP de seguridad.

HU-06 — Como desarrollador necesito catálogos de sesión sin consultas duplicadas.

Criterios de aceptación: Cinco catálogos cargados desde bootstrap; jerarquía por unidad_padre_id y limpieza al salir.

Estado: Implementado y validado

Evidencia y límites: login-contract.spec.ts: cinco catálogos, búsquedas, árbol, ruta y corte de ciclos.

HU-07 — Como usuario necesito recuperar un perfil válido al recargar.

Criterios de aceptación: Persistencia respaldada por backend, consulta /yo y recuperación sin confiar solo en JWT almacenado.

Estado: Pendiente

Evidencia y límites: Estado en memoria; recarga requiere nuevo login. No /yo ni persistencia segura confirmada.

HU-08 — Como mantenedor necesito documentación verificable del sprint.

Criterios de aceptación: Inventario, trazabilidad, evidencias de ejecución, TSDoc sin cambios funcionales y Word legible.

Estado: Implementado y validado

Evidencia y límites: Entregables presentes; igualdad del JavaScript transpilado y validación estructural del DOCX.

10.1 Matriz de trazabilidad

Historia	Requisito	Archivo	Endpoint	Prueba	Estado
HU-01	Login y bootstrap íntegro	LoginComponent; AuthService; validarLogin	/api/auth/token → /token	PV-01, PV-02	Implementado y validado
HU-01	Bearer y protección por sesión	authInterceptor; authGuard; AppShell	Backends configurados	PV-03, PV-04	Implementado y validado
HU-02	Rotación e inactividad 25/60	AuthService (sin implementación)	/api/v1/auth/refresh	PV-11	Pendiente
HU-03	Matriz numérica pura	authorization-policy.ts	Sin HTTP	PV-05	Implementado y validado
HU-03	Permisos efectivos por función	Sidebar; app.routes.ts; AuthService	Contrato/seed sin confirmar	Sin prueba operacional por ID	Parcialmente implementado
HU-04	Limpieza y aislamiento local	AuthService; AppShell; interceptores	Sin HTTP de logout	PV-04	Implementado y validado
HU-04	Revocación remota	Sin consumidor implementado	/api/v1/auth/logout	PV-11	Pendiente
HU-05	Consulta y desbloqueo IP	Sin módulo implementado	/api/v1/auth/seguridad[/bloqueo/{ip}]	PV-11	Pendiente
HU-06	Catálogos y jerarquía	CatalogosService; auth.models.ts	Bootstrap del login	PV-01	Implementado y validado
HU-07	Recuperación de perfil	Sin consumidor implementado	/api/v1/auth/yo	PV-11	Pendiente
HU-08	Documentación sin alterar lógica	13 TS comentados; 15 comparados; cinco entregables	OpenAPI solo lectura	PV-08, PV-10, PV-12	Implementado y validado

11. Pruebas y validación

Pruebas automatizadas ejecutadas durante este corte, con HttpTestingController y entorno de pruebas Angular/Vitest. Las pruebas de componentes y RouterTestingHarness validan integración interna con respuestas simuladas; no son una prueba funcional de FastAPI/Redis ni una validación manual en producción. No se registran tokens reales ni credenciales en los entregables.

ID	Caso	Esperado	Obtenido	Estado	Evidencia
PV-01	Contrato bootstrap y atomicidad	Rechazar JSON incompleto y cargar usuario/catálogos válidos	Selección enfocada aprobada; login-contract.spec.ts y api-connection.spec.ts	Implementado y validado	pruebas-autenticacion.log
PV-02	Login y mensajes	Evitar duplicados; manejar 401/422/red/500 sin detalles sensibles	Casos del componente aprobados con backend simulado	Implementado y validado	login.component.spec.ts; pruebas-autenticacion.log
PV-03	Headers y límites de backend	Bearer solo a destinos configurados; no refresh en solicitudes ordinarias	Casos API aprobados; no petición de refresh	Implementado y validado	api-connection.spec.ts; pruebas-autenticacion.log
PV-04	Guards y logout	Bloquear rutas tras salir, descartar respuestas antiguas, limpiar vistas privadas	Casos de logout/AppShell aprobados	Implementado y validado	logout.spec.ts; app-shell.spec.ts; pruebas-autenticacion.log
PV-05	Reglas numéricas	Aplicar matriz de roles y denegar IDs/contextos inválidos	Política pura aprobada; no demuestra protección de módulos	Implementado y validado	authorization-policy.spec.ts; pruebas-autenticacion.log
PV-06	Expiración y almacenamiento	Caducar según expira_en y no restaurar credenciales locales	Casos con reloj simulado aprobados	Implementado y validado	api-connection.spec.ts; login-contract.spec.ts
PV-07	Suite completa del repositorio	Todas las pruebas del estado actual deberían aprobar	166 aprobadas, 10 fallidas; 32 archivos aprobados y 5 fallidos, total 176/37	Parcialmente implementado	pruebas-completas.log
PV-08	Build desarrollo	Compilar TypeScript y templates Angular	Comando terminó correctamente	Implementado y validado	compilacion-desarrollo.log
PV-09	Build producción	Cumplir compilación y presupuestos	Bloqueado por CSS Hoja Milimetrada: 19,26 kB > 18 kB	Parcialmente implementado	compilacion-produccion.log
PV-10	Ausencia de cambio funcional en TSDoc	Conservar JavaScript transpilado en 15 archivos revisados	15/15 hashes idénticos con removeComments=true	Implementado y validado	verificacion-solo-comentarios.json
PV-11	Login/refresh/logout/seguridad reales v1	Contrato y comportamiento real de FastAPI/Redis	No ejecutados; endpoints ausentes del OpenAPI local	No verificable	contexto-revision.json
PV-12	Word	Paquete OOXML íntegro y apertura sin reparación	Abierto con Microsoft Word mediante COM; campos/páginas actualizados y PDF exportado; sin aprobación editorial	Implementado y validado	validacion-documento.json; PDF de presentación

11.1 Comandos y resultados

npm test -- --watch=false

npm test -- --watch=false --include=src/app/core/auth/*.spec.ts --include=src/app/core/services/api-connection.spec.ts --include=src/app/core/interceptors/*.spec.ts --include=src/app/pages/login/login.component.spec.ts --include=src/app/pages/sidebar/sidebar.spec.ts --include=src/app/layout/app-shell.spec.ts

npx ng build --configuration development

npx ng build

Resultado enfocado: 8 archivos y 53 pruebas aprobadas. Global: 37 archivos, 176 pruebas; 5 archivos con 10 fallos y 32 archivos aprobados con 166 casos. Desarrollo compila. Producción falla por hoja-milimetrada.component.css (19,26 kB frente a 18 kB) y registra avisos CommonJS de Leaflet/MapLibre. No se modifica ese módulo ni los budgets.

11.2 Fallos de la suite global

Archivo	Resultado observado
src/app/app.spec.ts	1 fallo: expectativa del título inicial.
src/app/pages/vulcano/menu-sistemas/menu-sistemas.spec.ts	2 fallos: IDs de trabajos esperados distintos del estado actual.
src/app/pages/vulcano/ventana-sistemas-artilleria/ventana-sistemas-artilleria.spec.ts	2 fallos: destinos/contenido provisional esperados.
src/app/pages/vulcano/hoja-milimetrada/proyectar.spec.ts	1 fallo: registrarCoordenadas no fue invocado según expectativa.
src/app/shared/components/mapa/mapa.spec.ts	4 fallos: casos de trayectoria/marcadores en Leaflet; ver log para pila completa.

Estos archivos no se modificaron en la fase documental. Los fallos se reportan tal como se ejecutaron; no se atribuyen a una causa única ni se ocultan con cambios de expectativas. La igualdad de JavaScript en el código comentado permite descartar alteraciones ejecutables introducidas por TSDoc.

12. Resultados y avances del sprint

Avance comprobado	Contribución / límite
Bootstrap estricto y centralizado	Impide publicar una sesión con usuario o catálogos incompletos.
Tokens solo en memoria y expiración local	Evita restauración automática de tokens persistidos antiguos; no equivale a revocación remota.
Generación local y cancelación de login	Reduce repoblación de estado privado por respuestas tardías.
Limpieza del shell y catálogos	Aísla vistas y datos entre sesiones.
Bearer limitado por destino	Evita enviar credenciales a dominios de mapas u otros orígenes no configurados.
Política numérica probada	Prepara reglas del contrato nuevo; conexión al producto pendiente.
Documentación institucional y TSDoc	Explica entradas, retornos y riesgos sin variar comportamiento.

El comparativo con el contrato anterior se sustenta en los informes locales y el árbol disponible. El historial Git no permite fechar cada mejora por separado; no se cuantifica productividad, velocidad del sprint o esfuerzo sin registros. El corte demuestra capacidades existentes y documentación generada, no la finalización de la arquitectura de seguridad v1.

12.1 Bitácora y evidencia Git

Commits disponibles: 961b545|2026-10-22T09:06:55-05:00|Excluir modelo OBJ pesado del repositorio; b33a1d2|2026-10-22T08:59:34-05:00|se sube todo todo el proyecto; 0630bd0|2026-08-10T14:08:50-05:00|initial commit. El historial sobre rutas auth/login/interceptores contiene b33a1d2. Hay cambios sin commit en el árbol; no se inventan autores o fechas de implementación. La bitácora distingue el historial de repositorio de las actividades de revisión realizadas hoy.

Fecha	Actividad	Resultado / evidencia
2026-10-22T09:06:55-05:00	Commit 961b545	Commit existente; no acredita sprint terminado — git log; contexto-revision.json
2026-10-22T08:59:34-05:00	Commit b33a1d2	Commit existente; no acredita sprint terminado — git log; contexto-revision.json
2026-08-10T14:08:50-05:00	Commit 0630bd0	Commit existente; no acredita sprint terminado — git log; contexto-revision.json
2026-10-27	Revisión técnica del árbol actual	Inventario y riesgos registrados — INVENTARIO_ARCHIVOS_AUTENTICACION.md
2026-10-27	Documentación TSDoc	JavaScript idéntico en 15/15 archivos — evidencias/verificacion-solo-comentarios.json
2026-10-27	Verificación Angular	Resultados reproducibles guardados — evidencias/*.log
2026-10-27	Generación documental	Archivos generados desde fuentes locales — generar_documentacion.py; evidencias

13. Riesgos y actividades pendientes

R-01 | Prioridad Alta

Hallazgo: Autorización operacional por IDs sin integrar: guards solo autentican y menús usan booleanos estáticos.

Impacto o limitación: Un usuario autenticado podría visualizar/activar funciones sin filtrado local por permisos; backend debe impedir operaciones no autorizadas.

Recomendación: Obtener seed/contrato, conectar política numérica a rutas/menús/acciones y probar cada función. No asumir una vulnerabilidad del backend sin evidencia.

R-02 | Prioridad Alta

Hallazgo: Logout únicamente local, sin revocación confirmada.

Impacto o limitación: No garantiza invalidación remota de un token previamente emitido.

Recomendación: Confirmar contrato logout y su semántica de revocación; diferenciar fallo remoto y cierre local.

R-03 | Prioridad Alta

Hallazgo: Sin refresh, control humano de actividad ni sesión deslizante.

Impacto o limitación: Interrumpe continuidad al vencer access token; requisito de inactividad no cubierto.

Recomendación: Implementar rotación single-flight y ventana de actividad tras confirmar contrato; nunca renovar indefinidamente por tráfico automático.

R-04 | Prioridad Media

Hallazgo: Expiración basada en expira_en local, sin lectura del JWT real.

Impacto o limitación: No detecta diferencias con exp o revocación antes de recibir rechazo del backend.

Recomendación: Confirmar claims y comparar vigencias sin intentar verificar firma con secretos en Angular; backend mantiene autoridad.

R-05 | Prioridad Media

Hallazgo: errorInterceptor registra HttpErrorResponse completo para 5xx.

Impacto o limitación: El cuerpo del servidor podría incluir datos sensibles en consola.

Recomendación: Redactar detalle, limitar logs por entorno y definir telemetría sin credenciales en un cambio funcional separado.

R-06 | Prioridad Media

Hallazgo: Signals de AuthService escribibles desde consumidores; modelos/catálogos no profundamente inmutables.

Impacto o limitación: Posibles inconsistencias de estado por escrituras externas o mutación accidental.

Recomendación: Encapsular estado con signals privados/asReadonly y devolver vistas inmutables en sprint posterior.

R-07 | Prioridad Media

Hallazgo: Recarga sin persistencia ni /yo.

Impacto o limitación: Se pierde sesión en memoria al recargar; no hay recuperación confirmada.

Recomendación: Definir mecanismo de persistencia respaldado por backend; no persistir refresh por conveniencia.

R-08 | Prioridad Media

Hallazgo: Transporte y bases de producción requieren revisión de despliegue.

Impacto o limitación: Direcciones localhost en servicios auxiliares no prueban HTTPS ni enrutamiento institucional.

Recomendación: Verificar Nginx/HTTPS, CORS y URLs por ambiente con equipo de backend; no inferir configuración del servidor a partir del proxy de desarrollo.

R-09 | Prioridad Media

Hallazgo: Suite global con fallos y build producción bloqueado por presupuesto CSS.

Impacto o limitación: Impide afirmar que el producto completo está validado o listo para publicar.

Recomendación: Corregir en tareas separadas las pruebas/causas y revisar presupuesto de estilos sin alterar esta documentación funcionalmente.

R-10 | Prioridad Baja

Hallazgo: Documentación histórica diverge de código: README menciona arranque de NavigationService y contrato antiguo.

Impacto o limitación: Confusión durante transferencia del conocimiento.

Recomendación: Usar este inventario como corte vigente y actualizar material histórico con aprobación de alcance. No atribuir auditoría activa no encontrada.

R-11 | Prioridad Media

Hallazgo: Ruta de diagnóstico de terreno fuera del guard de AppShell.

Impacto o limitación: Superficie accesible sin autenticación por configuración actual.

Recomendación: Confirmar con responsables si debe ser pública; no cambiar rutas en esta fase documental.

13.1 Plan del siguiente sprint

7. Confirmar OpenAPI v1, ejemplos anónimos de bootstrap/JWT, seed de permisos y configuración de proxy/HTTPS.
8. Adaptar el contrato numérico en el servicio central y conectar cada función a un ID documentado con controles de ruta y acción.
9. Implementar refresh rotativo compartido, un reintento máximo y actividad humana eficiente; pruebas de concurrencia e inactividad.
10. Integrar logout remoto con resultado explícito de revocación; definir recuperación /yo y catálogos tras recarga sin endpoints inventados.
11. Crear seguridad administrativa con guard de rol, confirmación de IP y actualización tras desbloqueo real.
12. Resolver fallos globales/build y probar contrato real con backend autorizado antes de declarar la integración terminada.

14. Conclusiones

VULCANO dispone de una base frontend comprobable para autenticación bootstrap y cierre local: estado central, validación integral, catálogos, expiración y limpieza privada. La documentación identifica sus componentes y aporta evidencias reproducibles de 53 casos enfocados aprobados. El módulo aún no cumple de extremo a extremo los requisitos de permisos numéricos operativos, sesión deslizante o administración Redis. Esas capacidades exigen contrato y desarrollo posterior; su ausencia no debe confundirse con una integración terminada.

La transferencia queda sustentada mediante Word, diagramas, inventario, trazabilidad y bitácora. Los comentarios conservan el JavaScript generado y no introducen cambios funcionales. La aceptación institucional requiere completar los campos de responsable/estado y revisar los hallazgos y resultados globales antes de planear el despliegue.

15. Anexos

15.1 Matriz de archivos y contratos públicos

F-01. LoginComponent

Ruta: src/app/pages/login/login.component.ts

Responsabilidad: Formulario reactivo de ingreso, estado de envío y mensajes seguros.

Dependencias y consumidores: FormBuilder, AuthService, Router, DestroyRef; plantilla y fondo TopographyBackground.

Endpoints: login(usuario,password) a través de AuthService; no HTTP directo.

Seguridad: Evita envíos duplicados; takeUntilDestroyed cancela al salir; finalize libera el botón.

Errores: 401/400: credenciales; 422: revisar datos; red/servidor: mensaje controlado; contrato o timeout: error genérico.

Observaciones: El diseño no cambia. recordarUsuario no implementa persistencia de sesión. La contraseña existe en el control del formulario durante su vida; no se almacena como dato de sesión.

Método	Entradas	Retorno
openLogin	Sin parámetros	void
closeLogin	Sin parámetros	void
togglePassword	Sin parámetros	void
login	Sin parámetros	void
usuarioInvalid	Sin parámetros	boolean
passwordInvalid	Sin parámetros	boolean
objetivosCount	Sin parámetros	number

F-02. AuthService

Ruta: src/app/core/auth/auth.service.ts

Responsabilidad: Fuente central de usuario, access token, expiración, refresh reservado, logout y generación local de sesión.

Dependencias y consumidores: HttpClient, ApiConfigService, Router, CatalogosService, RxJS; guards, interceptores, Sidebar y LoginComponent.

Endpoints: POST ApiConfigService.loginUrl, actualmente /api/auth/token. Ninguna solicitud de refresh, logout o perfil.

Seguridad: Tokens en memoria; validación completa previa; copia de usuario/catálogos; cancelación de login anterior; limpieza de claves antiguas; respuestas aisladas por generación.

Errores: Timeout de 30000 ms; error de login limpia estado sin cancelar su propia notificación. Fallos de limpieza/navegación se advierten y no restauran la sesión.

Observaciones: isAuthenticated revisa la fecha absoluta derivada de expira_en; no decodifica JWT. getCurrentUser/getRole no comprueban por sí mismos expiración. Mock preexistente solo si useMockAuth y no producción, actualmente desactivado. No se reproducen sus credenciales en este documento.

Método	Entradas	Retorno
sessionGeneration	Sin parámetros	number
registerSessionCleanup	cleanup: () => void	() => void

Método	Entradas	Retorno
constructor	private readonly http: HttpClient, private readonly apiConfig: ApiConfigService, private readonly router: Router, private readonly catalogos: CatalogosService	inferido por TypeScript
login	usuario: string, password: string	Observable<LoginResponse>
logout	Sin parámetros	Promise<void>
isAuthenticated	Sin parámetros	boolean
getToken	Sin parámetros	string | null
getApiKey	Sin parámetros	string | null
getCurrentUser	Sin parámetros	AuthUser | null
getRole	Sin parámetros	string | null
obtenerUsuario	Sin parámetros	AuthUser | null
obtenerNombreCompleto	Sin parámetros	string
obtenerRol	Sin parámetros	string | null
obtenerPermisos	Sin parámetros	string[]
tienePermiso	codigo: string	boolean
tieneRol	rol: string	boolean
obtenerFotoUsuario	Sin parámetros	string | null
loginWithMock	usuario: string, password: string	Observable<LoginResponse>
loginWithBackend	usuario: string, password: string	Observable<LoginResponse>
applySession	session: AuthSession	void
clearSession	redirectToLogin = false, cancelPending = true	void
mapLoginResponse	response: LoginResponse	AuthSession
canUseMockAuth	Sin parámetros	boolean
removeValue	key: string	void
clearAuthenticationCookies	Sin parámetros	void

F-03. Interfaces de autenticación

Ruta: src/app/core/auth/auth.models.ts

Responsabilidad: Define LoginResponse, AuthSession, UsuarioAutenticado/AuthUser, LoginRequest y los cinco catálogos.

Dependencias y consumidores: AuthService, validador, CatalogosService y pruebas.

Endpoints: Modelo del bootstrap consumido por login; no ejecuta HTTP.

Seguridad: Estructuras conocidas sin any; campos del backend conservados.

Errores: Las interfaces no validan JSON en ejecución: lo hace validarLogin.

Observaciones: El contrato activo exige usuario.permisos:string[], rol:string y expira_en:number. No existe JwtPayload activo. Los datos de tokens y usuario son campos separados, aunque todavía se exponen signals escribibles.

No se extraen métodos de clase de este archivo. Entradas y retornos de guards/interceptores se describen en su responsabilidad; las plantillas y configuraciones no son servicios HTTP.

F-04. validarLogin

Ruta: src/app/core/auth/auth-response.validator.ts

Responsabilidad: Acota una entrada no confiable a LoginResponse verificando todos los elementos del bootstrap.

Dependencias y consumidores: LoginResponse; invocado por mapLoginResponse antes de publicar estado.

Endpoints: Sin llamadas; procesa respuesta de login.

Seguridad: Tokens no vacíos, bearer, expiración positiva y finita; usuario y cinco arreglos tipados; IDs de catálogo no duplicados.

Errores: Lanza Error controlado ante datos faltantes o incompatibles; AuthService limpia la sesión.

Observaciones: No verifica firma, exp, sub, rol numérico ni revocación. No limita tamaño de arrays ni valida todas las reglas semánticas de datos de negocio.

Método	Entradas	Retorno
validarLogin	v: unknown	asserts v is LoginResponse

F-05. CatalogosService y NodoUnidad

Ruta: src/app/core/auth/catalogos.service.ts

Responsabilidad: Conserva bootstrap de catálogos y ofrece consultas de jerarquía.

Dependencias y consumidores: CatalogosLogin, UnidadMilitar, signals; AuthService carga y limpia.

Endpoints: Sin HTTP; no requiere endpoint adicional.

Seguridad: Copia al cargar y estado público de solo lectura a nivel de signal; ciclos detenidos mediante Set.

Errores: Búsquedas sin coincidencia devuelven undefined; rutas inexistentes devuelven arreglo vacío. structuredClone puede lanzar.

Observaciones: Usa unidad_padre_id para raíces, hijas, superior, árbol y ruta. Las huérfanas aparecen como raíces del árbol. Los objetos devueltos no están profundamente congelados.

Método	Entradas	Retorno
cargar	catalogos: CatalogosLogin	void
limpiar	Sin parámetros	void
unidades	Sin parámetros	UnidadMilitar[]
obtenerPrincipales	Sin parámetros	UnidadMilitar[]
obtenerSubordinadas	id: number	UnidadMilitar[]
buscarPorId	id: number	UnidadMilitar | undefined
buscarPorCodigo	codigo: string	UnidadMilitar | undefined
obtenerSuperior	id: number	UnidadMilitar | undefined
construirJerarquia	Sin parámetros	NodoUnidad[]
obtenerRuta	id: number	UnidadMilitar[]

F-06. Política numérica preparada

Ruta: src/app/core/auth/authorization-policy.ts

Responsabilidad: Reglas puras para ContextoAutorizacion con rol ADMINISTRADOR/USUARIO y permisos number[].

Dependencias y consumidores: Pruebas authorization-policy.spec.ts; sin dependencias Angular.

Endpoints: Sin HTTP ni conexión a JWT.

Seguridad: Administrador sin lista permite IDs válidos; demás casos exigen asignación; sesión ausente, rol inválido e IDs no positivos/enteros se deniegan.

Errores: Devuelve false, sin lanzar, para contextos inválidos; listas vacías de requisitos no autorizan.

Observaciones: NO se importa desde AuthService, rutas o menús. Las pruebas prueban la política aislada, no autorización operacional. No confundir con tienePermiso(codigo:string).

Método	Entradas	Retorno
tienePermisoNumerico	contexto: ContextoAutorizacion, id: number	boolean
tieneAlgunPermisoNumerico	contexto: ContextoAutorizacion, ids: readonly number[]	boolean
tieneTodosLosPermisosNumericos	contexto: ContextoAutorizacion, ids: readonly number[]	boolean

F-07. authGuard

Ruta: src/app/core/auth/auth.guard.ts

Responsabilidad: Protege las rutas hijas del AppShell por presencia de sesión local vigente.

Dependencias y consumidores: AuthService, Router, APP_PATHS; app.routes.ts.

Endpoints: Sin HTTP.

Seguridad: Retorna UrlTree a /login en lugar de navegar imperativamente.

Errores: No hay sesión: redirección; token vencido: AuthService limpia estado.

Observaciones: No comprueba rol ni permisos de una funcionalidad; no valida vigencia remota.

No se extraen métodos de clase de este archivo. Entradas y retornos de guards/interceptores se describen en su responsabilidad; las plantillas y configuraciones no son servicios HTTP.

F-08. guestGuard

Ruta: src/app/core/auth/guest.guard.ts

Responsabilidad: Reserva login a visitantes sin sesión local activa.

Dependencias y consumidores: AuthService, Router, APP_PATHS.

Endpoints: Sin HTTP.

Seguridad: Con sesión retorna UrlTree a /home.

Errores: Sin sesión permite acceso; la expiración se evalúa en AuthService.

Observaciones: No es un guard de administrador ni sustituye autorización del servidor.

No se extraen métodos de clase de este archivo. Entradas y retornos de guards/interceptores se describen en su responsabilidad; las plantillas y configuraciones no son servicios HTTP.

F-09. Contextos HTTP

Ruta: src/app/core/auth/auth-http-context.ts

Responsabilidad: Declara SKIP_AUTH_INTERCEPTOR y SKIP_ERROR_INTERCEPTOR con valor por defecto false.

Dependencias y consumidores: AuthService e interceptores auth/error.

Endpoints: Login utiliza ambos contextos de exclusión.

Seguridad: Evita adjuntar credenciales previas y evita logout global ante credenciales incorrectas.

Errores: No procesa errores; controla qué interceptor los atiende.

Observaciones: No existe contexto de reintento/refresh en esta versión.

No se extraen métodos de clase de este archivo. Entradas y retornos de guards/interceptores se describen en su responsabilidad; las plantillas y configuraciones no son servicios HTTP.

F-10. authInterceptor

Ruta: src/app/core/interceptors/auth.interceptor.ts

Responsabilidad: Adjunta Bearer a backends configurados, controla cache y filtra éxitos obsoletos.

Dependencias y consumidores: ApiConfigService, AuthService, SKIP_AUTH_INTERCEPTOR, RxJS filter.

Endpoints: Actúa en las bases configuradas; excluye token Bearer del login.

Seguridad: Comparación de origen y límite de ruta; cache:no-store en backends; proveedores externos no reciben credenciales.

Errores: No renueva ni reintenta; deja errores al siguiente interceptor. Solo filtra respuestas exitosas por generación.

Observaciones: X-API-Key se conserva como compatibilidad pero su signal queda null con el contrato vigente. No envía refresh token.

No se extraen métodos de clase de este archivo. Entradas y retornos de guards/interceptores se describen en su responsabilidad; las plantillas y configuraciones no son servicios HTTP.

F-11. errorInterceptor

Ruta: src/app/core/interceptors/error.interceptor.ts

Responsabilidad: Centraliza 401/403/5xx de los backends configurados.

Dependencias y consumidores: ApiConfigService, AuthService, SKIP_ERROR_INTERCEPTOR.

Endpoints: No crea solicitudes propias salvo logout local por AuthService.

Seguridad: 401 protegido solo afecta a la generación actual; login/exteriores excluidos; 403 no cierra sesión.

Errores: Rethrow de errores; 403 advierte; 5xx registra error completo en consola.

Observaciones: Hallazgo: HttpErrorResponse completo puede contener detalles sensibles del backend. Redactar logs en futuro sprint; no se cambia el comportamiento en esta fase.

No se extraen métodos de clase de este archivo. Entradas y retornos de guards/interceptores se describen en su responsabilidad; las plantillas y configuraciones no son servicios HTTP.

F-12. loaderInterceptor

Ruta: src/app/core/interceptors/loader.interceptor.ts

Responsabilidad: Activa LoaderService únicamente para solicitudes con LOADER_OPERATION.

Dependencias y consumidores: LoaderService; defer/finalize.

Endpoints: Transversal; no define endpoints.

Seguridad: No clasifica peticiones automáticas como actividad humana ni implementa temporizador de sesión.

Errores: finalize libera operación también en error/cancelación.

Observaciones: No sustituye el signal isSubmitting del login; incluido por su orden en la cadena global.

No se extraen métodos de clase de este archivo. Entradas y retornos de guards/interceptores se describen en su responsabilidad; las plantillas y configuraciones no son servicios HTTP.

F-13. ApiConfigService

Ruta: src/app/core/services/api-config.service.ts

Responsabilidad: Centraliza bases, URLs de login/logout/refresh y límites de destino.

Dependencias y consumidores: environment.api y environment.endpoints; AuthService e interceptores.

Endpoints: loginUrl=/api/auth/token; logoutUrl=/api/auth/logout y refreshTokenUrl=/api/auth/refresh configuradas pero no consumidas.

Seguridad: matchesBaseUrl verifica origin y frontera de pathname, evitando prefijos parecidos.

Errores: URL inválida produce false; base faltante utiliza respaldo auth.

Observaciones: joinUrl concatena, no descubre contrato ni convierte las rutas existentes a v1. La base de fotos se resuelve desde el origen auth.

Método	Entradas	Retorno
authBaseUrl	Sin parámetros	string
usuariosBaseUrl	Sin parámetros	string
archivosBaseUrl	Sin parámetros	string
operacionesBaseUrl	Sin parámetros	string
loginUrl	Sin parámetros	string
logoutUrl	Sin parámetros	string
refreshTokenUrl	Sin parámetros	string
isBackendUrl	url: string	boolean
isAuthEndpoint	url: string	boolean
isLoginEndpoint	url: string	boolean
getApiBaseUrl	service: keyof BackendMap	string
getBackendMap	Sin parámetros	BackendMap
getBackendBases	Sin parámetros	string[]
joinUrl	baseUrl: string, path: string	string
matchesBaseUrl	url: string, baseUrl: string	boolean

F-14. NavigationService

Ruta: src/app/core/services/navigation.service.ts

Responsabilidad: Preparado para registrar NavigationEnd y nombre de usuario en un signal en memoria.

Dependencias y consumidores: Router, AuthService, DestroyRef; RxJS y takeUntilDestroyed.

Endpoints: Sin HTTP.

Seguridad: No guarda tokens ni credenciales en sus entradas.

Errores: No constituye auditoría persistente ni demuestra eventos del servidor.

Observaciones: No se encontró inyección/instanciación en app.ts ni otros consumidores; no afirmar que el registro esté activo. Si se activa, definir retención y limpieza al cambiar de usuario.

Método	Entradas	Retorno
constructor	Sin parámetros	inferido por TypeScript

F-15. Sidebar y MenuItem

Ruta: src/app/pages/sidebar/sidebar.ts

Responsabilidad: Navegación de escritorio/móvil, visibilidad estática, cierre de menús y acción Salir.

Dependencias y consumidores: AuthService, ElementRef, RouterLink, RouterLinkActive, APP_PATHS.

Endpoints: Logout delegado a AuthService, actualmente solo local.

Seguridad: Salir es botón; cierra menús y utiliza cierre centralizado.

Errores: No hace HTTP ni asigna permisos ante errores.

Observaciones: canShowItem solo evalúa item.permiso !== false; permisoCodigo es metadato no aplicado. El menú no filtra por IDs del backend.

Método	Entradas	Retorno
toggleDropdown	name: string	void

Método	Entradas	Retorno
closeMenus	Sin parámetros	void
isDropdownOpen	name: string	boolean
onDropdownTriggerKeydown	event: KeyboardEvent, item: MenuItem	void
onDropdownLinkClick	Sin parámetros	void
toggleMobileMenu	Sin parámetros	void
onMobileNavigate	Sin parámetros	void
logout	Sin parámetros	void
canShowItem	item: MenuItem	boolean
getVisibleChildren	item: MenuItem	MenuItem[]
hasVisibleChildren	item: MenuItem	boolean
onDocumentClick	event: MouseEvent	void
onEscape	Sin parámetros	void

F-16. AppShell

Ruta: src/app/layout/app-shell.ts

Responsabilidad: Contenedor protegido y propietario del ciclo de vida de vistas/caché privada.

Dependencias y consumidores: AuthService, MapaElementosService, IconosService, MeasurementService, MapActionsService; PuntosDemoService por instancia.

Endpoints: No crea endpoints de autenticación.

Seguridad: Registra callback de limpieza; destruye vistas al navegar fuera y evita heredar caché de otro usuario.

Errores: AuthService aísla errores de callbacks; ngOnDestroy retira registro y limpia estado.

Observaciones: La documentación no altera herramientas geográficas ni sus algoritmos.

Método	Entradas	Retorno
clearPrivateState	Sin parámetros	void
ngOnDestroy	Sin parámetros	void

F-17. routes

Ruta: src/app/app.routes.ts

Responsabilidad: Define login con guestGuard y AppShell con canActivateChild authGuard.

Dependencias y consumidores: Componentes cargados de forma diferida y APP_ROUTES.

Endpoints: Rutas frontend; no son endpoints del backend.

Seguridad: Rutas Home, Ingreso, Permisos, Vulcano y VulcanoLog requieren autenticación.

Errores: Fallback a Home; guards generan redirección a login cuando corresponde.

Observaciones: La ruta diagnóstica de terreno está fuera del shell. No hay guard por IDs ni módulo de seguridad administrador.

No se extraen métodos de clase de este archivo. Entradas y retornos de guards/interceptores se describen en su responsabilidad; las plantillas y configuraciones no son servicios HTTP.

F-18. appConfig

Ruta: src/app/app.config.ts

Responsabilidad: Registra Router y HttpClient con interceptores funcionales.

Dependencias y consumidores: routes, loaderInterceptor, authInterceptor y errorInterceptor.

Endpoints: Sin rutas HTTP nuevas.

Seguridad: Orden de salida: loader → auth → error; las respuestas recorren la cadena inversa.

Errores: Registra escuchas de errores globales Angular.

Observaciones: No incorpora inicializador de recuperación de sesión ni /yo.

No se extraen métodos de clase de este archivo. Entradas y retornos de guards/interceptores se describen en su responsabilidad; las plantillas y configuraciones no son servicios HTTP.

F-19. sharedEnvironment

Ruta: src/environments/environment.shared.ts

Responsabilidad: Define timeout, bases de servicios, endpoints y useMockAuth=false.

Dependencias y consumidores: environment.ts y environment.production.ts lo extienden.

Endpoints: Base auth /api/auth; sufijos /token, /logout, /refresh.

Seguridad: No contiene secreto de firma JWT. El modo mock requiere además no producción.

Errores: No verifica conectividad ni despliegue.

Observaciones: Otros servicios conservan direcciones localhost: verificar configuración de producción y transporte HTTPS en despliegue real, sin inferir que ya estén expuestos.

No se extraen métodos de clase de este archivo. Entradas y retornos de guards/interceptores se describen en su responsabilidad; las plantillas y configuraciones no son servicios HTTP.

F-20. Proxy de desarrollo

Ruta: proxy.conf.cjs

Responsabilidad: Reenvía /api/auth/** al backend local y elimina el prefijo /api/auth.

Dependencias y consumidores: ng serve; backend de desarrollo.

Endpoints: /api/auth/token se traduce a /token en http://127.0.0.1:8010.

Seguridad: Configuración de desarrollo, no evidencia del proxy o HTTPS en producción.

Errores: Errores de conexión llegan al HttpClient/formulario.

Observaciones: No configura /api/v1/auth/**. Los contratos v1 requieren verificar también el proxy, no solo cambiar strings.

No se extraen métodos de clase de este archivo. Entradas y retornos de guards/interceptores se describen en su responsabilidad; las plantillas y configuraciones no son servicios HTTP.

F-21. STORAGE_KEYS

Ruta: src/app/core/constants/storage-keys.constants.ts

Responsabilidad: Enumera claves propias antiguas de token, usuario, clave API y expiración.

Dependencias y consumidores: AuthService y pruebas de aislamiento.

Endpoints: Sin HTTP.

Seguridad: Se usan para retirar solo datos de autenticación propios en ambos almacenamientos.

Errores: AuthService captura errores de acceso al storage.

Observaciones: La existencia de claves no significa que se escriban credenciales: la sesión actual solo vive en memoria.

No se extraen métodos de clase de este archivo. Entradas y retornos de guards/interceptores se describen en su responsabilidad; las plantillas y configuraciones no son servicios HTTP.

F-22. APP_ROUTES y APP_PATHS

Ruta: src/app/core/constants/routes.constants.ts

Responsabilidad: Nombres relativos y absolutos de navegación.

Dependencias y consumidores: Guards, Router, Sidebar, LoginComponent y AuthService.

Endpoints: No define API REST.

Seguridad: Mantiene las redirecciones centralizadas.

Errores: No maneja errores de red.

Observaciones: No documenta equivalencias entre funciones e IDs de permisos.

No se extraen métodos de clase de este archivo. Entradas y retornos de guards/interceptores se describen en su responsabilidad; las plantillas y configuraciones no son servicios HTTP.

F-23. Vista de login

Ruta: src/app/pages/login/login.component.html

Responsabilidad: Presenta controles, envío, errores y contenidos institucionales existentes.

Dependencias y consumidores: LoginComponent, formularios reactivos y TopographyBackground.

Endpoints: Sin HTTP directo.

Seguridad: Consume estados y validación del componente; no muestra tokens.

Errores: Muestra loginError controlado por el componente.

Observaciones: No se modifica el diseño en este trabajo; CSS se revisa como referencia de paleta.

No se extraen métodos de clase de este archivo. Entradas y retornos de guards/interceptores se describen en su responsabilidad; las plantillas y configuraciones no son servicios HTTP.

F-24. Vista de navegación

Ruta: src/app/pages/sidebar/sidebar.html

Responsabilidad: Muestra grado, nombre completo, rol y acciones de salida.

Dependencias y consumidores: Sidebar y AuthService.

Endpoints: Sin HTTP directo.

Seguridad: No imprime tokens; navegación y salida delegadas.

Errores: Sin procesamiento de respuestas.

Observaciones: Información de identidad visible, sin implementar una vista /yo ni panel de seguridad.

No se extraen métodos de clase de este archivo. Entradas y retornos de guards/interceptores se describen en su responsabilidad; las plantillas y configuraciones no son servicios HTTP.

15.2 Soporte, configuración y pruebas

Ruta	Responsabilidad / alcance
src/environments/environment.ts	Extiende sharedEnvironment con production:false.
src/environments/environment.production.ts	Extiende sharedEnvironment con production:true.
src/app/pages/login/login.component.css	Referencia visual: fondo oscuro y tonos oliva; no se modifica.
src/app/pages/sidebar/sidebar.css	Diseño de navegación; no se modifica.
src/app/app.ts	Raíz con RouterOutlet; no instancia NavigationService.
src/app/core/auth/auth-test.fixture.ts	Bootstrap sintético para pruebas, sin credenciales reales; no es respuesta de producción.

Ruta	Responsabilidad / alcance
src/app/core/auth/login-contract.spec.ts	Usuario, permisos textuales, cinco catálogos, jerarquía, errores y atomicidad.
src/app/core/auth/logout.spec.ts	Cierre local, cancelación, aislamiento entre sesiones y limpieza.
src/app/core/auth/authorization-policy.spec.ts	Matriz numérica aislada e IDs inválidos.
src/app/core/services/api-connection.spec.ts	URL-encoded, expiración, guard, destinos HTTP y respuestas 401/403.
src/app/core/interceptors/loader.interceptor.spec.ts	Liberación de operaciones del loader, también en error/cancelación.
src/app/pages/login/login.component.spec.ts	Envío único, navegación y errores seguros del formulario.
src/app/pages/sidebar/sidebar.spec.ts	Renderizado y acciones de salida de escritorio/móvil.
src/app/layout/app-shell.spec.ts	Vistas protegidas y limpieza de datos al cambiar usuario.
angular.json	Builders, proxy de serve y presupuestos de compilación.
package.json	Dependencias y comandos npm; versiones instaladas en contexto-revision.json.
src/app/core/auth/README.md	Material histórico; contrastar con código y este informe antes de reutilizar.
docs/login-contrato-fastapi.md	Informe anterior de adaptación; no sustituye la ejecución de pruebas de este corte.
docs/autenticacion-v1-pendientes.md	Preparación de política numérica y contratos v1 pendientes.

15.3 Fragmentos relevantes sin credenciales

Extractos de AuthService y política preparada, incluidos para aclarar la diferencia de contratos:

```
obtenerPermisos(): string[] { return [...(this.currentUser()?.permisos ?? [])]; }
```

```
tienePermiso(codigo: string): boolean { return this.isAuthenticated() &&
this.obtenerPermisos().includes(codigo); }
```

```
return (contexto.rol === 'ADMINISTRADOR' && contexto.permisos.length === 0) ||
contexto.permisos.includes(id);
```

```
expiresAt: Date.now() + response.expira_en * 1000
```

15.4 Evidencias y referencias locales

ARQUITECTURA_AUTENTICACION.md; MATRIZ_TRAZABILIDAD_AUTENTICACION.md; BITACORA_DESARROLLO_AUTENTICACION.md; INVENTARIO_ARCHIVOS_AUTENTICACION.md; evidencias/contexto-revision.json; evidencias/inventario-simbolos.json; evidencias/verificacion-solo-comentarios.json; evidencias/pruebas-autenticacion.log; evidencias/pruebas-completas.log; evidencias/compilacion-desarrollo.log; evidencias/compilacion-produccion.log; evidencias/validacion-documento.json

Fuentes primarias: código frontend revisado, árbol/historial Git y OpenAPI local consultado sin credenciales. Las especificaciones de Redis/JWT v1 son requerimientos recibidos y no una auditoría del backend. No se incorporan datos personales de usuarios reales ni secretos.